

```
std::forward<string>(string("the end"))
```

`forward<string>`的结果类型是 `string&&`，因此 `construct` 将得到一个右值引用实参。`construct` 会继续将此实参传递给 `string` 的移动构造函数来创建新元素。

706

建议：转发和可变参数模板

可变参数函数通常将它们的参数转发给其他函数。这种函数通常具有与我们的 `emplace_back` 函数一样的形式：

```
// fun 有零个或多个参数，每个参数都是一个模板参数类型的右值引用
template<typename... Args>
void fun(Args&&... args) // 将 Args 扩展为一个右值引用的列表
{
    // work 的实参既扩展 Args 又扩展 args
    work(std::forward<Args>(args)...);
}
```

这里我们希望将 `fun` 的所有实参转发给另一个名为 `work` 的函数，假定由它完成函数的实际工作。类似 `emplace_back` 中对 `construct` 的调用，`work` 调用中的扩展既扩展了模板参数包也扩展了函数参数包。

由于 `fun` 的参数是右值引用，因此我们可以传递给它任意类型的实参；由于我们使用 `std::forward` 传递这些实参，因此它们的所有类型信息在调用 `work` 时都会得到保持。

16.4.3 节练习

练习 16.58: 为你的 `StrVec` 类及你为 16.1.2 节（第 591 页）练习中编写的 `Vec` 类添加 `emplace_back` 函数。

练习 16.59: 假定 `s` 是一个 `string`，解释调用 `svec.emplace_back(s)` 会发生什么。

练习 16.60: 解释 `make_shared`（参见 12.1.1 节，第 401 页）是如何工作的。

练习 16.61: 定义你自己版本的 `make_shared`。



16.5 模板特例化

编写单一模板，使之对任何可能的模板实参都是最适合的，都能实例化，这并不总是能办到。在某些情况下，通用模板的定义对特定类型是不适合的：通用定义可能编译失败或做得不正确。其他时候，我们也可以利用某些特定知识来编写更高效的代码，而不是从通用模板实例化。当我们不能（或不希望）使用模板版本时，可以定义类或函数模板的一个特例化版本。

我们的 `compare` 函数是一个很好的例子，它展示了函数模板的通用定义不适合一个特定类型（即字符指针）的情况。我们希望 `compare` 通过调用 `strcmp` 比较两个字符指针而非比较指针值。实际上，我们已经重载了 `compare` 函数来处理字符串字面常量（参见 16.1.1 节，第 579 页）：

```
// 第一个版本; 可以比较任意两个类型
template <typename T> int compare(const T&, const T&);
// 第二个版本处理字符串字面常量
template<size_t N, size_t M>
int compare(const char (&)[N], const char (&)[M]);
```

707

但是, 只有当我们传递给 `compare` 一个字符串字面常量或者一个数组时, 编译器才会调用接受两个非类型模板参数的版本。如果我们传递给它字符指针, 就会调用第一个版本:

```
const char *p1 = "hi", *p2 = "mom";
compare(p1, p2);           // 调用第一个模板
compare("hi", "mom");      // 调用有两个非类型参数的版本
```

我们无法将一个指针转换为一个数组的引用, 因此当参数是 `p1` 和 `p2` 时, 第二个版本的 `compare` 是不可行的。

为了处理字符指针 (而不是数组), 可以为第一个版本的 `compare` 定义一个模板特例化 (template specialization) 版本。一个特例化版本就是模板的一个独立的定义, 在其中一个或多个模板参数被指定为特定的类型。

定义函数模板特例化

当我们特例化一个函数模板时, 必须为原模板中的每个模板参数都提供实参。为了指出我们正在实例化一个模板, 应使用关键字 `template` 后跟一个空尖括号对 (`<>`)。空尖括号指出我们将为原模板的所有模板参数提供实参:

```
// compare 的特殊版本, 处理字符数组的指针
template <>
int compare(const char* const &p1, const char* const &p2)
{
    return strcmp(p1, p2);
}
```

理解此特例化版本的困难之处是函数参数类型。当我们定义一个特例化版本时, 函数参数类型必须与一个先前声明的模板中对应的类型匹配。本例中我们特例化:

```
template <typename T> int compare(const T&, const T&);
```

其中函数参数为一个 `const` 类型的引用。类似类型别名, 模板参数类型、指针及 `const` 之间的相互作用会令人惊讶 (参见 2.5.1 节, 第 60 页)。

我们希望定义此函数的一个特例化版本, 其中 `T` 为 `const char*`。我们的函数要求一个指向此类型 `const` 版本的引用。一个指针类型的 `const` 版本是一个常量指针而不是指向 `const` 类型的指针 (参见 2.4.2 节, 第 56 页)。我们需要在特例化版本中使用的类型是 `const char* const &`, 即一个指向 `const char` 的 `const` 指针的引用。

函数重载与模板特例化

708

当定义函数模板的特例化版本时, 我们本质上接管了编译器的工作。即, 我们为原模板的一个特殊实例提供了定义。重要的是要弄清: 一个特例化版本本质上是一个实例, 而非函数名的一个重载版本。



特例化的本质是实例化一个模板, 而非重载它。因此, 特例化不影响函数匹配。

我们将一个特殊的函数定义为一个特例化版本还是一个独立的非模板函数，会影响到函数匹配。例如，我们已经定义了两个版本的 `compare` 函数模板，一个接受数组引用参数，另一个接受 `const T&`。我们还定义了一个特例化版本来处理字符指针，这对函数匹配没有影响。当我们对字符串字面常量调用 `compare` 时

```
compare("hi", "mom")
```

对此调用，两个函数模板都是可行的，且提供同样好的（即精确的）匹配。但是，接受字符串数组参数的版本更特例化（参见 16.3 节，第 615 页），因此编译器会选择它。

如果我们将接受字符指针的 `compare` 版本定义为一个普通的非模板函数（而不是模板的一个特例化版本），此调用的解析就会不同。在此情况下，将会有三个可行的函数：两个模板和非模板的字符指针版本。所有三个函数都提供同样好的匹配。如前所述，当一个非模板函数提供与函数模板同样好的匹配时，编译器会选择非模板版本（参见 16.3 节，第 615 页）。

关键概念：普通作用域规则应用于特例化

为了特例化一个模板，原模板的声明必须在作用域中。而且，在任何使用模板实例的代码之前，特例化版本的声明也必须在作用域中。

对于普通类和函数，丢失声明的情况（通常）很容易发现——编译器将不能继续处理我们的代码。但是，如果丢失了一个特例化版本的声明，编译器通常可以用原模板生成代码。由于在丢失特例化版本时编译器通常会实例化原模板，很容易产生模板及其特例化版本声明顺序导致的错误，而这种错误又很难查找。

如果一个程序使用一个特例化版本，而同时原模板的一个实例具有相同的模板实参集合，就会产生错误。但是，这种错误编译器又无法发现。

Best Practices

模板及其特例化版本应该声明在同一个头文件中。所有同名模板的声明应该放在前面，然后是这些模板的特例化版本。

709 类模板特例化

除了特例化函数模板，我们还可以特例化类模板。作为一个例子，我们将为标准库 `hash` 模板定义一个特例化版本，可以用它来将 `Sales_data` 对象保存在无序容器中。默认情况下，无序容器使用 `hash<key_type>`（参见 11.4 节，第 394 页）来组织其元素。为了让我们自己的数据类型也能使用这种默认组织方式，必须定义 `hash` 模板的一个特例化版本。一个特例化 `hash` 类必须定义：

- 一个重载的调用运算符（参见 14.8 节，第 506 页），它接受一个容器关键字类型的对象，返回一个 `size_t`。
- 两个类型成员，`result_type` 和 `argument_type`，分别调用运算符的返回类型和参数类型。
- 默认构造函数和拷贝赋值运算符（可以隐式定义，参见 13.1.2 节，第 443 页）。

在定义此特例化版本的 `hash` 时，唯一复杂的地方是：必须在原模板定义所在的命名空间中特例化它。我们将在 18.2 节（第 695 页）中介绍更多命名空间的相关内容。现在，我们只需知道——我们可以向命名空间添加成员。为了达到这一目的，首先必须打开命名空间：

```
// 打开 std 命名空间，以便特例化 std::hash
namespace std {
```

```
} // 关闭 std 命名空间；注意：右花括号之后没有分号
```

花括号对之间的任何定义都将成为命名空间 std 的一部分。

下面的代码定义了一个能处理 Sales_data 的特例化 hash 版本：

```
// 打开 std 命名空间，以便特例化 std::hash
namespace std {
template <> // 我们正在定义一个特例化版本，模板参数为 Sales_data
struct hash<Sales_data>
{
    // 用来散列一个无序容器的类型必须要定义下列类型
    typedef size_t result_type;
    typedef Sales_data argument_type; // 默认情况下，此类型需要==
    size_t operator()(const Sales_data& s) const;
    // 我们的类使用合成的拷贝控制成员和默认构造函数
};
size_t
hash<Sales_data>::operator()(const Sales_data& s) const
{
    return hash<string>()(s.bookNo) ^
           hash<unsigned>()(s.units_sold) ^
           hash<double>()(s.revenue);
}
} // 关闭 std 命名空间；注意：右花括号之后没有分号
```

我们的 hash<Sales_data> 定义以 template<> 开始，指出我们正在定义一个全特例化的模板。我们正在特例化的模板名为 hash，而特例化版本为 hash<Sales_data>。接下来的类成员是按照特例化 hash 的要求而定义的。

710

类似其他任何类，我们可以在类内或类外定义特例化版本的成员，本例中就是在类外定义的。重载的调用运算符必须为给定类型的值定义一个哈希函数。对于一个给定值，任何时候调用此函数都应该返回相同的结果。一个好的哈希函数对不相等的对象（几乎总是）应该产生不同的结果。

在本例中，我们将定义一个好的哈希函数的复杂任务交给了标准库。标准库为内置类型和很多标准库类型定义了 hash 类的特例化版本。我们使用一个（未命名的）hash<string> 对象来生成 bookNo 的哈希值，用一个 hash<unsigned> 对象来生成 units_sold 的哈希值，用一个 hash<double> 对象来生成 revenue 的哈希值。我们将这些结果进行异或运算（参见 4.8 节，第 137 页），形成给定 Sales_data 对象的完整的哈希值。

值得注意的是，我们的 hash 函数计算所有三个数据成员的哈希值，从而与我们为 Sales_data 定义的 operator==（参见 14.3.1 节，第 497 页）是兼容的。默认情况下，为了处理特定关键字类型，无序容器会组合使用 key_type 对应的特例化 hash 版本和 key_type 上的相等运算符。

假定我们的特例化版本在作用域中，当将 Sales_data 作为容器的关键字类型时，编译器就会自动使用此特例化版本：

```
// 使用 hash<Sales_data> 和 14.3.1 节（第 497 页）中 Sales_data 的 operator==
unordered_multiset<Sales_data> SDset;
```

由于 hash<Sales_data> 使用 Sales_data 的私有成员，我们必须将它声明为

Sales_data 的友元:

```
template <class T> class std::hash; // 友元声明所需要的
class Sales_data {
    friend class std::hash<Sales_data>;
    // 其他成员定义, 如前
};
```

这段代码指出特殊实例 hash<Sales_data>是 Sales_data 的友元。由于此实例定义在 std 命名空间中, 我们必须记得在 friend 声明中应使用 std::hash。



为了让 Sales_data 的用户能使用 hash 的特例化版本, 我们应该在 Sales_data 的头文件中定义该特例化版本。

类模板部分特例化

与函数模板不同, 类模板的特例化不必为所有模板参数提供实参。我们可以只指定一部分而非所有模板参数, 或是参数的一部分而非全部特性。一个类模板的**部分特例化** (partial specialization) 本身是一个模板, 使用它时用户还必须为那些在特例化版本中未指定的模板参数提供实参。



我们只能部分特例化类模板, 而不能部分特例化函数模板。

在 16.2.3 节 (第 605 页) 中我们介绍了标准库 remove_reference 类型。该模板是通过一系列的特例化版本来完成其功能的:

```
// 原始的、最通用的版本
template <class T> struct remove_reference {
    typedef T type;
};
// 部分特例化版本, 将用于左值引用和右值引用
template <class T> struct remove_reference<T&> // 左值引用
{ typedef T type; };
template <class T> struct remove_reference<T&&> // 右值引用
{ typedef T type; };
```

第一个模板定义了最通用的模板。它可以用任意类型实例化; 它将模板实参作为 type 成员的类型。接下来的两个类是原始模板的部分特例化版本。

由于一个部分特例化版本本质是一个模板, 与往常一样, 我们首先定义模板参数。类似任何其他特例化版本, 部分特例化版本的名字与原模板的名字相同。对每个未完全确定类型的模板参数, 在特例化版本的模板参数列表中都有一项与之对应。在类名之后, 我们为要特例化的模板参数指定实参, 这些实参列于模板名之后的尖括号中。这些实参与原始模板中的参数按位置对应。

部分特例化版本的模板参数列表是原始模板的参数列表的一个子集或者是一个特例化版本。在本例中, 特例化版本的模板参数的数目与原始模板相同, 但是类型不同。两个特例化版本分别用于左值引用和右值引用类型:

```
int i;
// decltype(42) 为 int, 使用原始模板
remove_reference<decltype(42)>::type a;
```

```
// decltype(i) 为 int&, 使用第一个 (T&) 部分特例化版本
remove_reference<decltype(i)>::type b;
// decltype(std::move(i)) 为 int&&, 使用第二个 (即 T&&) 部分特例化版本
remove_reference<decltype(std::move(i))>::type c;
```

三个变量 a、b 和 c 均为 int 类型。

特例化成员而不是类

我们可以只特例化特定成员函数而不是特例化整个模板。例如, 如果 Foo 是一个模板类, 包含一个成员 Bar, 我们可以只特例化该成员:

712

```
template <typename T> struct Foo {
    Foo(const T &t = T()): mem(t) { }
    void Bar() { /* ... */ }
    T mem;
    // Foo 的其他成员
};

template<> // 我们正在特例化一个模板
void Foo<int>::Bar() // 我们正在特例化 Foo<int> 的成员 Bar
{
    // 进行应用于 int 的特例化处理
}
```

本例中我们只特例化 Foo<int> 类的一个成员, 其他成员将由 Foo 模板提供:

```
Foo<string> fs; // 实例化 Foo<string>::Foo()
fs.Bar(); // 实例化 Foo<string>::Bar()
Foo<int> fi; // 实例化 Foo<int>::Foo()
fi.Bar(); // 使用我们特例化版本的 Foo<int>::Bar()
```

当我们用 int 之外的任何类型使用 Foo 时, 其成员像往常一样进行实例化。当我们用 int 使用 Foo 时, Bar 之外的成员像往常一样进行实例化。如果我们使用 Foo<int> 的成员 Bar, 则会使用我们定义的特例化版本。

16.5 节练习

练习 16.62: 定义你自己版本的 hash<Sales_data>, 并定义一个 Sales_data 对象的 unordered_multiset。将多条交易记录保存到容器中, 并打印其内容。

练习 16.63: 定义一个函数模板, 统计一个给定值在一个 vector 中出现的次数。测试你的函数, 分别传递给它一个 double 的 vector, 一个 int 的 vector 以及一个 string 的 vector。

练习 16.64: 为上一题中的模板编写特例化版本来处理 vector<const char*>。编写程序使用这个特例化版本。

练习 16.65: 在 16.3 节 (第 617 页) 中我们定义了两个重载的 debug_rep 版本, 一个接受 const char* 参数, 另一个接受 char* 参数。将这两个函数重写为特例化版本。

练习 16.66: 重载 debug_rep 函数与特例化它相比, 有何优点和缺点?

练习 16.67: 定义特例化版本会影响 debug_rep 的函数匹配吗? 如果不影响, 为什么?

713 小结

模板是 C++ 语言与众不同的特性，也是标准库的基础。一个模板就是一个编译器用来生成特定类类型或函数的蓝图。生成特定类或函数的过程称为实例化。我们只编写一次模板，就可以将其用于多种类型和值，编译器会为每种类型和值进行模板实例化。

我们既可以定义函数模板，也可以定义类模板。标准库算法都是函数模板，标准库容器都是类模板。

显式模板实参允许我们固定一个或多个模板参数的类型或值。对于指定了显式模板实参的模板参数，可以应用正常的类型转换。

一个模板特例化就是一个用户提供的模板实例，它将一个或多个模板参数绑定到特定类型或值上。当我们不能（或不希望）将模板定义用于某些特定类型时，特例化非常有用。

最新 C++ 标准的一个主要部分是可变参数模板。一个可变参数模板可以接受数目和类型可变的参数。可变参数模板允许我们编写像容器的 `emplace` 成员和标准库 `make_shared` 函数这样的函数，实现将实参传递给对象的构造函数。

术语表

类模板 (class template) 模板定义，可从它实例化出特定的类。类模板的定义以关键字 `template` 开始，后跟尖括号对 `<` 和 `>`，其内为一个用逗号分隔的一个或多个模板参数的列表，随后是类的定义。

默认模板实参 (default template argument) 一个类型或一个值，当用户未提供对应模板实参时，模板会使用它。

显式实例化 (explicit instantiation) 一个声明，为所有模板参数提供了显式实参。714 实例 (instantiation) 编译器从模板生成用来指导实例化过程。如果声明是 `extern` 的，模板将不会被实例化；否则，模板将利用指定的实参进行实例化。对每个 `extern` 模板声明，在程序中某处必须有一个非 `extern` 的显式实例化。

显式模板实参 (explicit template argument) 在一个函数调用中或定义模板类类型时，由用户提供的模板实参。显式模板实参在紧跟在模板名的尖括号对中给出。

函数参数包 (function parameter pack) 表示零个或多个函数参数的参数包。

函数模板 (function template) 模板定义，可从它实例化出特定函数。函数模板的定

义以关键字 `template` 开始，后跟尖括号对 `<` 和 `>`，其内为一个用逗号分隔的一个或多个模板参数的列表，随后是函数的定义。

实例化 (instantiate) 编译器处理过程，用实际的模板实参来生成模板的一个特殊实例，其中参数被替换为对应的实参。当函数模板被调用时，会自动根据传递给它的实参来实例化。而使用类模板时，则需要我们提供显式模板实参。

实例 (instantiation) 编译器从模板生成的类或函数。

成员模板 (member template) 本身是模板的成员函数。成员模板不能是虚函数。

非类型参数 (nontype parameter) 表示值的模板参数。非类型模板参数的实参必须是常量表达式。

包扩展 (pack expansion) 处理过程，将一个参数包替换为其中元素的列表。

参数包 (parameter pack) 表示零个或多个参数的模板或函数参数。

部分特例化 (partial specialization) 类模板的一个版本，其中指定了某些但不是所